Chatty

Java Multi-Client Chat Application

Technical Report & Code Documentation

Java 17+ Swing GUI TCP Sockets Multi-threaded

1. Project Overview

Chatty is a real-time, multi-client chat application written entirely in Java. It consists of a TCP socket server that manages concurrent user sessions and a Swing-based graphical client that lets users connect, exchange messages, and inspect who is online — all without any third-party networking framework.

Key capabilities at a glance:

■ **Real-time messaging** Messages are broadcast to every connected client instantly via a shared ConcurrentHashMap of PrintWriter streams.

■ **Concurrent connections** Each client is handled by a dedicated thread (ClientHandler), allowing many users simultaneously.

■ **Username registration** The server enforces unique, non-empty usernames before a client may send messages.

■ **Active user list** Typing AllUsers returns a live roster; the GUI sidebar displays it as labelled entries.

■ **Graceful disconnect** Sending Bye triggers a clean broadcast and removes the user from the active map.

■ **Swing GUI** Start.java provides a themed login screen; Dashboard.java hosts the chat area, sidebar, and input field.

2. Architecture

2.1 High-Level Design

The system follows a client–server architecture utilizing TCP. The server runs a single accept-loop and spawns one thread per client. The GUI client connects once, sends the chosen username, then enters a message loop driven by the Swing event-dispatch thread.

CLIENT SIDE		SERVER SIDE
`Start.java (Login GUI)`	■■■■■■■■■■■■ ■■■■ ■■■■■■■■ ■■■■■■■■	`Server.java (ServerSocket :1234)`
`ChatClient.java (Socket wrapper)`	TCP :1234	`ClientHandler (one thread/client)`
`Dashboard.java (Chat GUI)`		`ConcurrentHashMap (username→writer)`

2.2 Threading Model

The server's main thread blocks on `server.accept()`. Each accepted socket is wrapped in a ClientHandler and handed to a new Thread. On the client side,`ChatClient.startListening()` creates a daemon thread

that reads lines from the socket and hands them to the Swing EDT via `SwingUtilities.invokeLater()`, ensuring thread safety.

3. Source File Breakdown

Server.java [SERVER]

Houses the entire server logic. Opens a ServerSocket on port 1234, maintains a ConcurrentHashMap<String, PrintWriter> of active clients, and spawns a ClientHandler thread for every accepted connection.

clients (static)	Thread-safe map from username to output stream.
broadcast()	Iterates the map and prints a timestamped line to every client.
activeUsers()	Returns a comma-separated string of all current usernames.
ClientHandler.run()	Handles username registration then enters the command/message loop.

ChatClient.java [CLIENT LIB]

Thin wrapper around a java.net.Socket. Provides connect(), send(), startListening(), and close() methods. The GUI never touches raw sockets directly.

connect()	Opens socket, wraps streams in PrintWriter / BufferedReader.
send(String msg)	Writes a line to the server; no-op if not connected.
startListening(Consumer)	Starts a daemon thread; delivers each line to the callback on the EDT.
close()	Closes all streams and the socket gracefully.

Start.java [GUI]

The entry point and login screen. Collects server IP, port, and username. On connect it instantiates ChatClient, sends the username, then opens Dashboard.

connect()	Validates fields, creates ChatClient, sends username, opens Dashboard.
main()	Invokes Start on the Swing EDT via SwingUtilities.invokeLater().

Dashboard.java [GUI]

The main chatwindow. Contains a top bar, a collapsible sidebar with a user list, a scrollable chat area, and an input field. Registers a message listener on ChatClient.

startListening callback	Parses 'Active users ->' lines into sidebar labels; appends everything else to chatArea.
sendMessage()	Reads the input field, optionally shows the sidebar, and calls client.send().
toggleUserList()	Flips the sidebar panel's visibility and revalidates.
updateUserListFromServerLine()	Splits the user list string and rebuilds JLabel entries in userListPanel.

Client.java [CLI]

A command-line client is provided as a simple alternative to the GUI. Reads host/port from args, spawns a reader thread, and echoes stdin to the server.

reader thread	Prints every server line to stdout.
main loop	Reads scanner lines and sends them until 'exit' is typed.

4. Protocol & Message Flow

All communication is plain-text, new-line-delimited over a single TCP stream. The table below lists every command and system message:

Direction	Message / Command	Description
S → C	`Server: Username =`	Prompts the client to enter a username
C → S	`<username>`	Client's chosen display name
S → all	`Server: Welcome <username>`	Broadcast when user joins
C → S	`AllUsers`	Request the active user list
S → C	`Server: Active users -> a, b, c`	Comma-separated list of online users
C → S	`<any text>`	Chat message; broadcast as username: text
C → S	`Bye`	Graceful disconnect; broadcasts goodbye
S → all	`Server: Goodbye <username>`	Broadcast on disconnect

4.1 Connection Sequence

```
Client Server
| |
  |--- TCP connect() ------------>|
  |<-- "Server: Username =" ------|
  |--- <username> -------------->|  |
  <-- "Server: Welcome alice" ---|
| (broadcast to ALL) |
  |--- <message> ------------->|
  |<-- "alice: hello" ------------|
| (broadcast to ALL) |
  |--- Bye ---------------------->|
  |<-- "Server: Goodbye alice" ---|
| (broadcast to ALL) |
  |--- TCP close() -------------->|
```

5. GUI Walkthrough

5.1 Login Screen (Start.java)

When launched, the application displays a themed login window (520 × 360 px). A background image (bck.png) fills the frame. The card panel collects three fields: Server IP (default localhost), Port (default 1234), and Username. Pressing Enter in any field or clicking Connect validates the inputs and attempts a socket connection.

Figure 1 — Login screen mockup

5.2 Dashboard (Dashboard.java)

After a successful connection the login window is disposed and the dashboard opens. It is a 900 × 600 px BorderLayout frame divided into four zones:

Top bar (NORTH)	Dark header showing 'Welcome, <username>' in white bold.
Sidebar (WEST)	180 px wide; contains an 'All Users' button and a collapsible user-list panel.
Chat area (CENTER)	Black JTextArea with white text, wrapped in a JScrollPane.
Input panel (SOUTH)	45 px tall; JTextField fires sendMessage() on Enter.

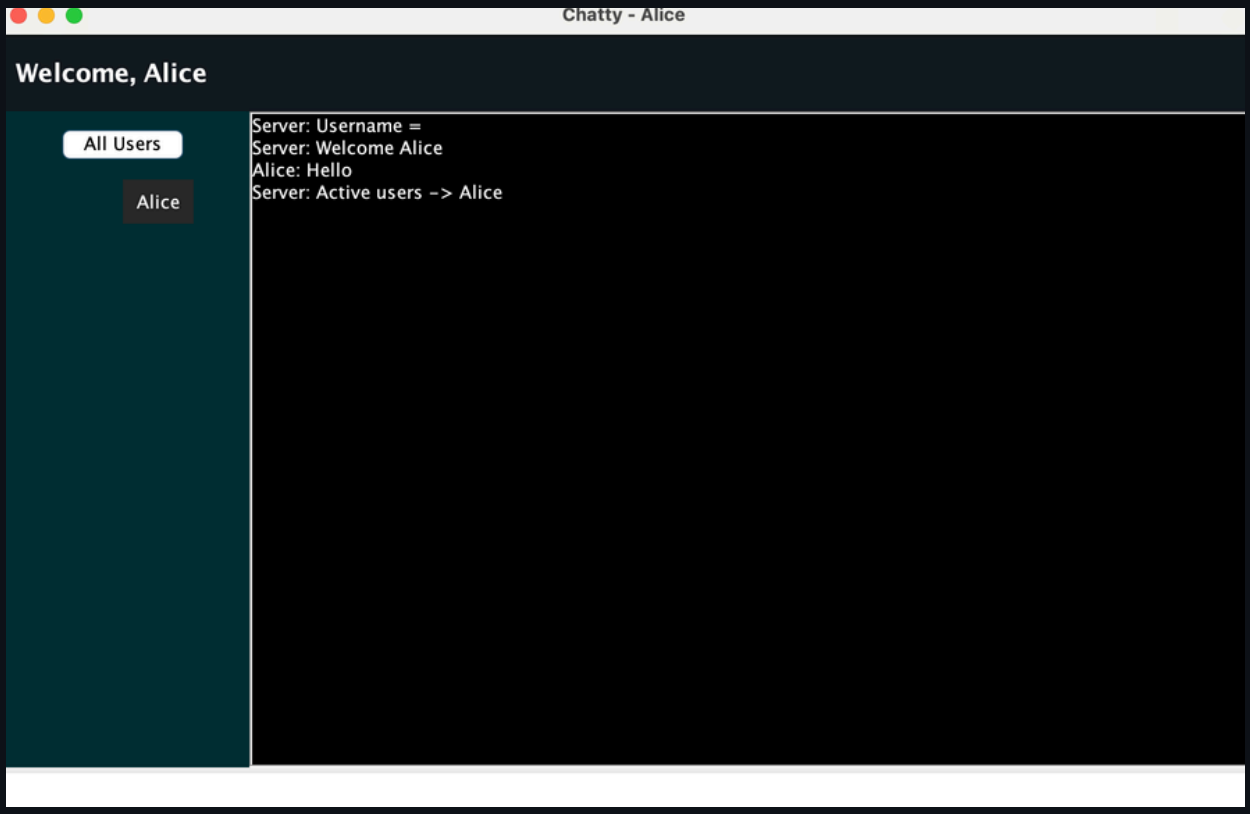

Figure 2 — Dashboard mockup (sidebar expanded, 3 users online)

6. How to Compile & Run

Step 1 — Compile all sources

```
javac Server.java ChatClient.java Dashboard.java Start.java Client.java
```

Step 2 — Start the server

```
java Server
```

Step 3 — Launch the GUI client (repeat for each user)

```
java Start
```

Step 4 (optional) — CLI client

```
java Client localhost 1234
```

Note: All five .java files must reside in the same directory. The server must be started before any client attempts to connect. The background image bck.png should be placed in the same directory as the compiled classes for the login screen to display correctly.

7. Known Bugs/Future Updates

Type	Issue	Details
Bug	**Client.java ignores the host/port from args**	Line 17 hard-codes localhost:1234 instead of using the parsed args[0] and args[1].
UX	**Login screen sends username before handshake**	Start.java sends the username immediately after connect(); if the server is slow the prompt may not have arrived yet.
Feature	**No private messaging**	All messages are global broadcasts; a /dm command could route to a single PrintWriter.
Feature	**No message history**	Clients joining late see no previous messages; a scrollback buffer on the server could fix this.

Security	**Plaintext protocol**	All traffic is unencrypted; wrapping the socket in SSLSocket would address this.
Robustness	**No heartbeat / timeout**	A disconnected client without sending Bye leaves a stale entry in the map until the next write fails.

8. Conclusion

Chatty illustrates Java networking and GUI concepts in a self-contained codebase. The server-side concurrency model — one thread per client plus a thread-safe broadcast map — is known in Java and scales well for a small number of users. The Swing client separates the socket layer (ChatClient) from presentation (Dashboard), making it straightforward to swap in a different transport or UI toolkit in the future. The issues catalogued in Section 7 provide a roadmap for extending the application.

End of Report